MIPS ASSEMBLY LANGUAGE PROGRAM

CONTENTS

01. MIPS assembly language programming

02. Tools

03. Program template, directives,text, data segments

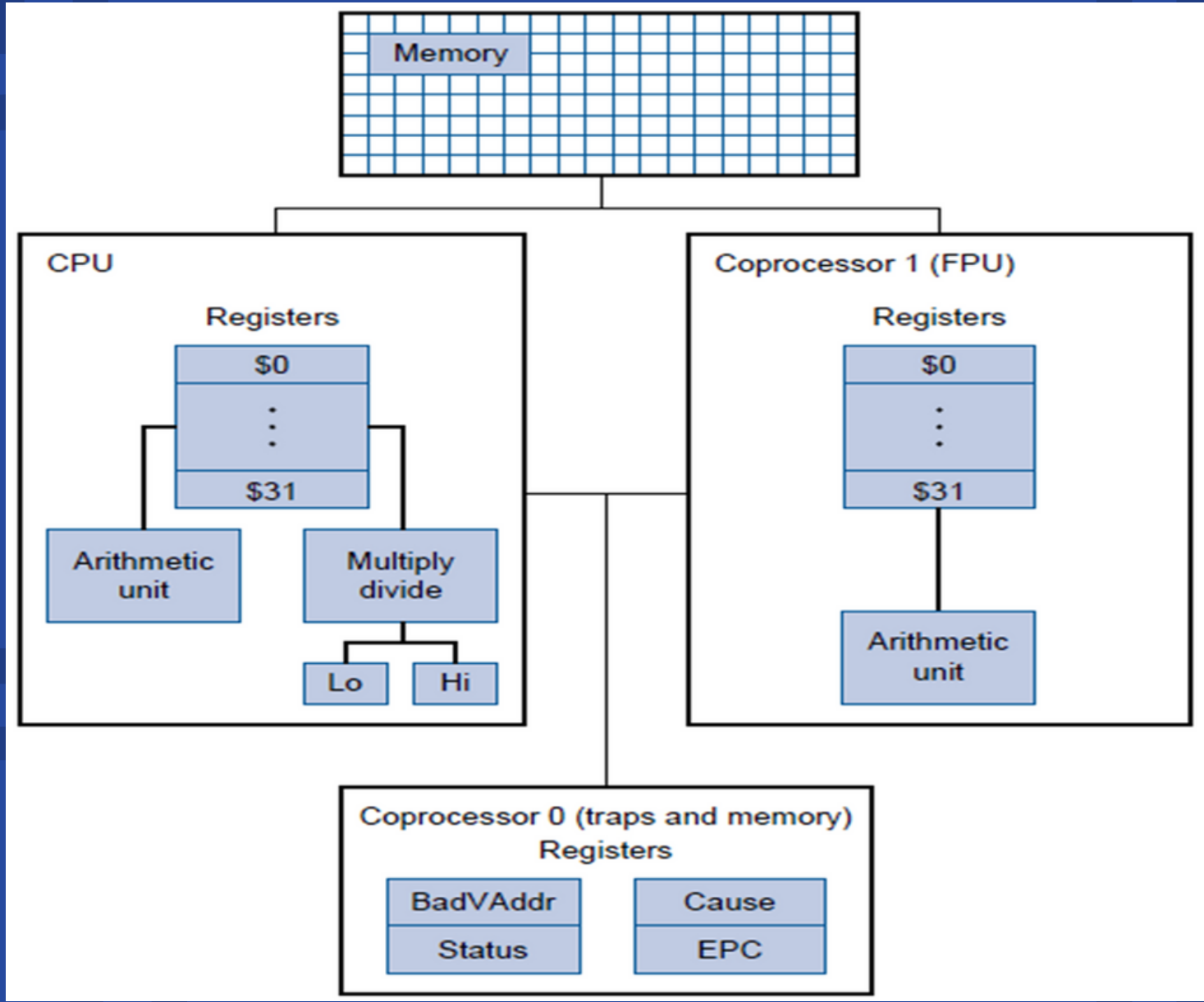
04. Procedure calls, runtime stack and stack pointer, stack segments

MIPS ASSEMBLY LANGUAGE PROGRAM

MIPS assembly language is a low-level programming language used for the MIPS architecture. Understanding MIPS assembly is essential for system programming, embedded systems, and performance optimization. In this presentation, we'll explore the basics, procedures, control flow, and more.

Let's proceed!

MIPS ARCHITECTURE



MIPS INSTRUCTION SET

Arithmetic and Logical Instructions

Instruction	Operation
add $d, $s, $t	$d = $s + $t
addu $d, $s, $t	$d = $s + $t
addi $t, $s, i	$t = $s + SE(i)
addiu $t, $s, i	$t = $s + SE(i)
and $d, $s, $t	$d = $s & $t
andi $t, $s, i	$t = $s & ZE(i)
div $s, $t	lo = $s / $t; hi = $s % $t
divu $s, $t	lo = $s / $t; hi = $s % $t
mult $s, $t	hi:lo = $s * $t
multu $s, $t	hi:lo = $s * $t
nor $d, $s, $t	$d = ~($s | $t)
or $d, $s, $t	$d = $s | $t
ori $t, $s, i	$t = $s | ZE(i)
sll $d, $t, a	$d = $t << a
sllv $d, $t, $s	$d = $t << $s
sra $d, $t, a	$d = $t >> a
srav $d, $t, $s	$d = $t >> $s
srl $d, $t, a	$d = $t >>> a
srlv $d, $t, $s	$d = $t >>> $s
sub $d, $s, $t	$d = $s - $t
subu $d, $s, $t	$d = $s - $t
xor $d, $s, $t	$d = $s ^ $t
xori $d, $s, i	$d = $s ^ ZE(i)

Constant-Manipulating Instructions

Instruction	Operation
lhi $t, i	HH($t) = i
llo $t, i	LH($t) = i

Comparison Instructions

Instruction	Operation
slt $d, $s, $t	$d = ($s < $t)
sltu $d, $s, $t	$d = ($s < $t)
slti $t, $s, i	$t = ($s < SE(i))
sltiu $t, $s, i	$t = ($s < SE(i))

Branch Instructions

Instruction	Operation
beq $s, $t, label	if ($s == $t) pc += i << 2
bgtz $s, label	if ($s > 0) pc += i << 2
blez $s, label	if ($s <= 0) pc += i << 2
bne $s, $t, label	if ($s != $t) pc += i << 2

Jump Instructions

Instruction	Operation
j label	pc += i << 2
jal label	$31 = pc; pc += i << 2
jalr $s	$31 = pc; pc = $s
jr $s	pc = $s

Load Instructions

Instruction	Operation
lb $t, i($s)	$t = SE (MEM [$s + i]:1)
lbu $t, i($s)	$t = ZE (MEM [$s + i]:1)
lh $t, i($s)	$t = SE (MEM [$s + i]:2)
lhu $t, i($s)	$t = ZE (MEM [$s + i]:2)
lw $t, i($s)	$t = MEM [$s + i]:4

Store Instructions

Instruction	Operation
sb $t, i($s)	MEM [$s + i]:1 = LB ($t)
sh $t, i($s)	MEM [$s + i]:2 = LH ($t)
sw $t, i($s)	MEM [$s + i]:4 = $t

Data Movement Instructions

Instruction	Operation
mfhi $d	$d = hi
mflo $d	$d = lo
mthi $s	hi = $s
mtlo $s	lo = $s

Exception and Interrupt Instructions

Instruction	Operation
trap 1	Print integer value in $4
trap 5	Read integer value into $2
trap 10	Terminate program execution
trap 101	Print ASCII character in $4
trap 102	Read ASCII character into $2

TOOLS

Moving forward, let's discuss tools for MIPS programming.
Let's proceed!

TOOLS

- An **emulator** is hardware or software or both that duplicates (or *emulates*) the functions of one computer system (the *guest*) in another computer system (the *host*).

MARS
(MIPS ASSEMBLER AND RUNTIME SIMULATOR)

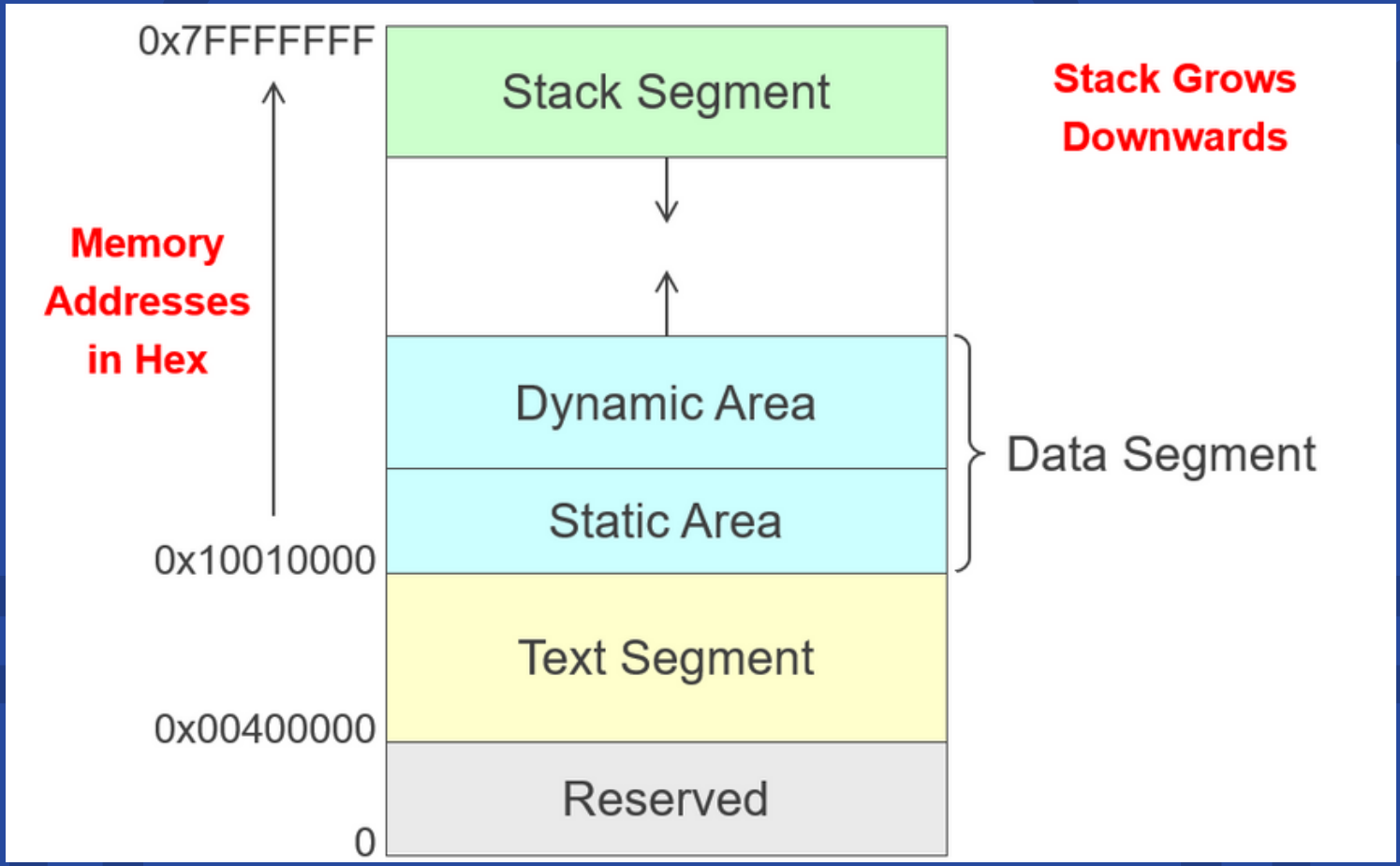
PROGRAM TEMPLATE

This section will cover the program template. In MIPS assembly, a program template is a structured framework for organizing your code. It provides the essential sections and directives for a MIPS assembly program. Using a template ensures consistency, readability, and maintainability of your code. Let's explore further!

PROGRAM TEMPLATE

```
1  # Program Name: YourProgramName
2  # Description: Brief description of your program
3
4  # .data section
5  .data
6      # Define your data variables here using directives like .word, .byte, etc.
7
8
9  # .text section
10 .text
11     # Main program code starts here
```

MIPS MEMORY LAYOUT

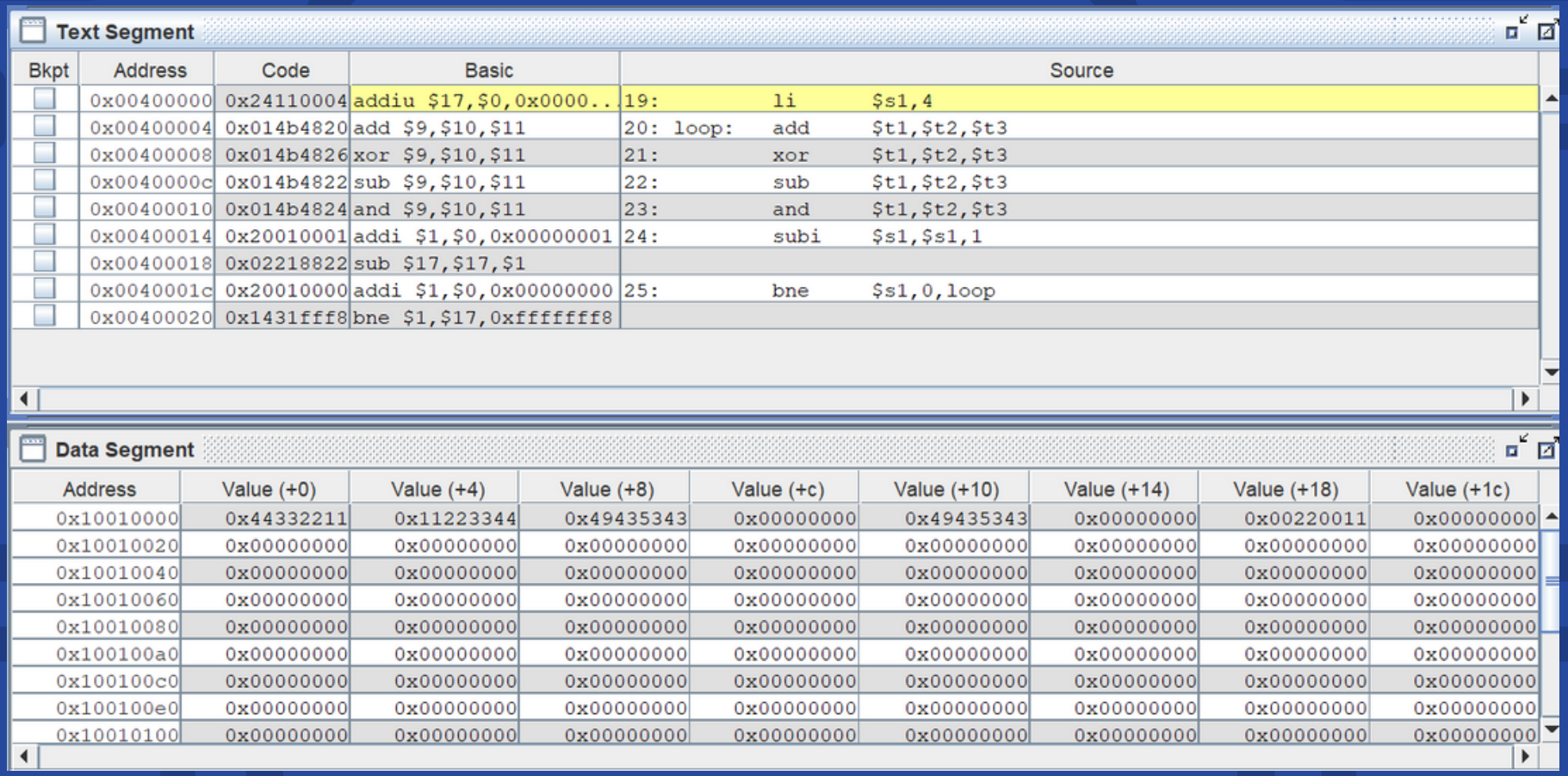


MIPS PROGRAM STRUCTURE

```
# .data section
        .data
    # Define your data variables here using directives like .word, .byte, etc.
var1:   .byte 0x11,0x22,0x33,0x44
var2:   .word 0x11223344
var3:   .asciiz "CSCI"
        .align 2
var4:   .ascii "CSCI"
        .space 4
var5:   .half 0x11,0x22


# .text section
        .text
    # Main program code starts here
        li      $s1,4
loop:   add     $t1,$t2,$t3
        xor     $t1,$t2,$t3
        sub     $t1,$t2,$t3
        and     $t1,$t2,$t3
        subi    $s1,$s1,1
        bne     $s1,0,loop
```

MIPS PROGRAM STRUCTURE



Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x24110004	addiu $17,$0,0x0000..	19: li $s1,4
	0x00400004	0x014b4820	add $9,$10,$11	20: loop: add $t1,$t2,$t3
	0x00400008	0x014b4826	xor $9,$10,$11	21: xor $t1,$t2,$t3
	0x0040000c	0x014b4822	sub $9,$10,$11	22: sub $t1,$t2,$t3
	0x00400010	0x014b4824	and $9,$10,$11	23: and $t1,$t2,$t3
	0x00400014	0x20010001	addi $1,$0,0x00000001	24: subi $s1,$s1,1
	0x00400018	0x02218822	sub $17,$17,$1	
	0x0040001c	0x20010000	addi $1,$0,0x00000000	25: bne $s1,0,loop
	0x00400020	0x1431fff8	bne $1,$17,0xfffffff8	

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x44332211	0x11223344	0x49435343	0x00000000	0x49435343	0x00000000	0x00220011	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

DIRECTIVES

.align	Align next data item on specified byte boundary (0=byte, 1=half, 2=word, 3=double)
.ascii	Store the string in the Data segment but do not add null terminator
.asciiz	Store the string in the Data segment and add null terminator
.byte	Store the listed value(s) as 8 bit bytes
.data	Subsequent items stored in Data segment at next available address
.double	Store the listed value(s) as double precision floating point
.end_macro	End macro definition. See .macro
.eqv	Substitute second operand for first. First operand is symbol, second operand is expression (like #define)
.extern	Declare the listed label and byte length to be a global data field
.float	Store the listed value(s) as single precision floating point
.globl	Declare the listed label(s) as global to enable referencing from other files
.half	Store the listed value(s) as 16 bit halfwords on halfword boundary
.include	Insert the contents of the specified file. Put filename in quotes.
.kdata	Subsequent items stored in Kernel Data segment at next available address
.ktext	Subsequent items (instructions) stored in Kernel Text segment at next available address
.macro	Begin macro definition. See .end_macro
.set	Set assembler variables. Currently ignored but included for SPIM compatability
.space	Reserve the next specified number of bytes in Data segment
.text	Subsequent items (instructions) stored in Text segment at next available address
.word	Store the listed value(s) as 32 bit words on word boundary

PROCEDURE CALLS

Let's dive into procedure. A procedure is a named block of code that performs a specific task. Let's get started!

PROCEDURE

- A procedure (or function) is a tool used by programmers
  - Allows the programmer to focus on just one task at a time
  - Allows code to be reused
- Procedure Call and Return
  - Put parameters in a place where procedure can access
    - Four argument registers: **`$a0`** thru **`$a3`** in which to pass parameters
  - Transfer control to the procedure and save return address
    - Jump-and-Link instruction: **`jal`** (Return Address saved in **`$ra`**)
  - Perform the desired task
  - Put results in a place where the calling procedure can access
    - Two value registers to return results: **$v0** and **$v1**
  - Return to calling procedure: **`jr $ra`** (jump to return address)

INSTRUCTIONS FOR PROCEDURE

- JAL (Jump-and-Link) used as the call instruction
  - Save return address in **$ra = PC+4** and jump to procedure
  - Register **$ra = $31** is used by **JAL** as the **return address**
- JR (Jump Register) used to return from a procedure
  - Jump to instruction whose address is in register Rs (PC = Rs)
- JALR (Jump-and-Link Register)
  - Save return address in Rd = PC+4, and
  - Jump to procedure whose address is in register Rs (PC = Rs)
  - Can be used to call methods (addresses known only at runtime)

Instruction		Meaning	Format					
jal	label	$31=PC+4, jump	$op^6 = 3$	imm^{26}				
jr	Rs	PC = Rs	$op^6 = 0$	rs^5	0	0	0	8
jalr	Rd, Rs	Rd=PC+4, PC=Rs	$op^6 = 0$	rs^5	0	rd^5	0	9

CALLER SHOULD:

1. Pass Arguments
   - First four arguments are passed in registers $a0 thru $a3
   - Additional arguments are pushed on the stack memory
2. Save Registers $a0 - $a3 and $t0 - $t9 if needed
   - Registers $a0 - $a3 and $t0 - $t9 should be saved by Caller
   - To preserve their value if needed after a procedure call
   - Called procedure is free to modify $a0 to $a3 and $t0 to $t9
3. Execute JAL Instruction
   - Jumps to the first instruction inside the procedure
   - Saves the return address in register $ra

CALLEE SHOULD:

1. Allocate memory for the stack frame
   - $sp = $sp – n (n bytes are allocated on the stack frame)
   - The programmer should compute n
   - A simple leaf procedure might not need a stack frame (n = 0)
2. Save registers $ra, $fp, $s0 - $s7 in the stack frame
   - $ra, $fp, $s0 - $s7 should be saved inside procedure (callee)
   - Before modifying their value and only if needed
   - $ra should be saved only if the procedure makes a call
3. Update the frame pointer $fp (if needed)
   - For simple procedures, the $fp register is not be required

PROCEDURE EXAMPLE

```
main:
    addi $a1, $zero, 50
    addi $a2, $zero, 100
    jal  addNumbers

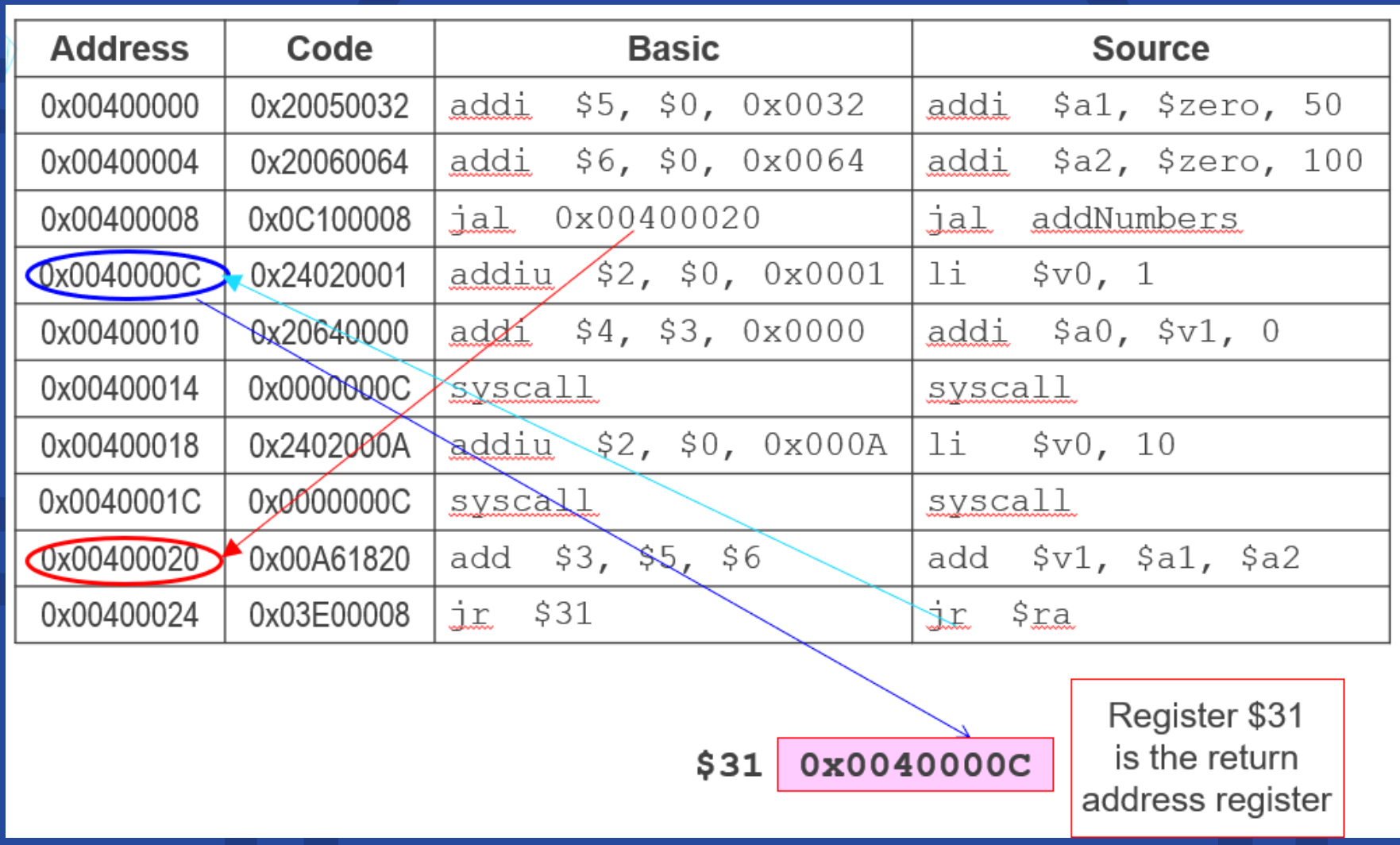
    li   $v0, 1
    addi $a0, $v1, 0
    syscall

    # Tell the system that the program is done
    li   $v0, 10
    syscall

addNumbers:
    add  $v1, $a1, $a2
    jr   $ra
```

PROCEDURE EXAMPLE

Address	Code	Basic	Source
0x00400000	0x20050032	addi $5, $0, 0x0032	addi $a1, $zero, 50
0x00400004	0x20060064	addi $6, $0, 0x0064	addi $a2, $zero, 100
0x00400008	0x0C100008	jal 0x00400020	jal addNumbers
0x0040000C	0x24020001	addiu $2, $0, 0x0001	li $v0, 1
0x00400010	0x20640000	addi $4, $3, 0x0000	addi $a0, $v1, 0
0x00400014	0x0000000C	syscall	syscall
0x00400018	0x2402000A	addiu $2, $0, 0x000A	li $v0, 10
0x0040001C	0x0000000C	syscall	syscall
0x00400020	0x00A61820	add $3, $5, $6	add $v1, $a1, $a2
0x00400024	0x03E00008	jr $31	jr $ra



RUNTIME STACK

- The runtime stack is a critical data structure used in program execution and memory management.
- It serves as a temporary storage area for data associated with procedure calls.
- Facilitates the management of control flow and data within a program.
- The runtime stack operates on the "last-in, first-out" (LIFO) principle.
- It is used for both data storage and controlling procedure calls.

RUNTIME STACK

- When a procedure is called, a new stack frame is created, representing that procedure's execution context.
- Data relevant to the procedure call, including the return address, parameters, and local variables, is stored within this frame.

RUNTIME STACK

- Data relevant to the procedure call, including the return address, parameters, and local variables, is stored within this frame.
- Push: Adding data to the top of the stack.
- Pop: Removing data from the top of the stack.
- Peek: Viewing the data at the top without removing it.

RUNTIME STACK

```
1  .data
2  output: .asciiz "Result: "
3
4  .text
5  .globl main
6
7  main:
8      # Initialize variables
9      li $a0, 5       # Set an argument
10     jal factorial   # Call the factorial procedure
11     move $s0, $v0   # Save the result in a register
12
13     # Print the result
14     li $v0, 4       # Load print string syscall code
15     la $a0, output  # Load the address of the output string
16     syscall
17
18     move $a0, $s0   # Load the result to print
19     li $v0, 1        # Load print integer syscall code
20     syscall
21
22     # Exit the program
23     li $v0, 10
24     syscall
```

RUNTIME STACK

```
26  factorial:
27      # Stack frame setup
28      sub $sp, $sp, 8  # Allocate space on the stack for a stack frame
29      sw $ra, 4($sp)  # Save the return address on the stack
30
31      # Procedure logic
32      beqz $a0, base_case  # If the argument is 0, return 1
33      subi $sp, $sp, 4  # Allocate space for a local variable
34      sw $a0, 0($sp)  # Save the argument on the stack
35      subi $a0, $a0, 1  # Decrement the argument
36      jal factorial  # Recursive call
37      lw $a0, 0($sp)  # Restore the original argument
38      mul $v0, $v0, $a0  # Multiply the result by the original argument
39      addi $sp, $sp, 4  # Deallocate space for the local variable
40      j done
41
42  base_case:
43      li $v0, 1  # Return 1 for the base case
44
45  done:
46      # Stack frame cleanup
47      lw $ra, 4($sp)  # Restore the return address
48      addi $sp, $sp, 8  # Deallocate the stack frame
49
50      jr $ra  # Return to the caller
```

STACK POINTER

- A stack frame is a dedicated portion of the runtime stack associated with a single procedure call.
- It contains critical data and control information.

STACK FRAME

Components of a Stack Frame

- Return Address: The address to which control should be returned upon exiting the procedure.
- Parameters: Values passed to the procedure.
- Local Variables: Variables declared within the procedure.

STACK FRAME

Components of a Stack Frame

- Saved Registers: Registers that need to be preserved across procedure calls.
- Additional Data: Exception handlers, debug information, etc.

STACK FRAME

- Organisation
  - A stack frame is organised as a contiguous block of memory, typically with the return address at the top.
  - Local variables and parameters are stored below the return address.
  - Saved registers may be present at the bottom of the frame.

STACK FRAME

- Creation
  - When a procedure is called, a new stack frame is created on top of the stack.
  - The return address is pushed onto the new frame, followed by parameters and local variables.
- Deallocation
  - Stack frames are deallocated upon returning from a procedure call.
  - The top frame is removed, and control returns to the caller's frame.

NESTED PROCEDURE

```
1   main:
2       # ... main program code ...
3       jal outer_procedure
4       # ... more main program code ...
5
6   outer_procedure:
7       # ... outer procedure code ...
8       jal inner_procedure
9       # ... more outer procedure code ...
10
11  inner_procedure:
12      # ... inner procedure code ...
13      jr $ra  # Return from inner procedure
```

NESTED PROCEDURE

```
1   outer_procedure:
2       sub $sp, $sp, 12
3       sw $ra, 8($sp)
4       sw $s0, 4($sp)
5       move $s0, $a0
6       jal inner_procedure
7       move $a0, $s0
8       lw $s0, 4($sp)
9       lw $ra, 8($sp)
10      add $sp, $sp, 12
11      jr $ra
12
13  inner_procedure:
14      sub $sp, $sp, 8
15      sw $ra, 4($sp)
16      # Inner procedure code
17      lw $ra, 4($sp)
18      add $sp, $sp, 8
19      jr $ra
```